Caves

DESCUBRÍ LO QUE PASA CERCA TUYO

Reporte Técnico del Proyecto

Arquitectura, Decisiones y Progreso

Cliente: CavesApp

Elaborado por: Craftia — Consultoría de IA y Desarrollo

Fecha: 24 de marzo de 2026

Horas invertidas: 8 horas

Versión: 1.0 — MVP Fase 1

Tabla de Contenidos

01 Resumen Ejecutivo 3

02 Descripción del Producto 3

03 Arquitectura y Stack Tecnológico 4

04 Supabase vs Firebase: Decisión Técnica 5

05 Arquitectura por Features (Screaming Architecture) 6

06 Modelo de Datos 7

07 Estrategia de Testing 8

08 Progreso Actual y Roadmap 9

09 Desglose de Inversión de Tiempo 10

01 Resumen Ejecutivo

Caves es una plataforma de descubrimiento de eventos basada en geolocalización, orientada al mercado mexicano (demográfico 18-35 años). Los usuarios navegan flyers de eventos a pantalla completa, ordenados por proximidad — como un mapa social visual donde los eventos flotan alrededor de tu ubicación. El MVP se construye como una PWA mobile-first con Next.js 15, Supabase y un lenguaje de diseño dark-first.

73	168	17	8h
ARCHIVOS FUENTE	TESTS UNITARIOS	SPECS E2E	HORAS INVERTIDAS

02 Descripción del Producto

Audiencia Objetivo

SEGMENTO	DESCRIPCIÓN	PROPUESTA DE VALOR
Asistentes	Jóvenes de 18-35 años buscando vida nocturna, conciertos, ferias gastronómicas	Un swipe para ver todo lo que hay cerca esta noche
Promotores	Organizadores de eventos buscando visibilidad local	Visibilidad gratuita, segmentada por ubicación
Venues	Bares, clubs, galerías, restaurantes	Plataforma de promoción recurrente de eventos

Funcionalidades Core (MVP)

FUNCIONALIDAD	ESTADO	DESCRIPCIÓN
Feed de Flyers	LISTO	Grilla a pantalla completa con swipe horizontal, ordenamiento por proximidad
Sistema de Auth	LISTO	Email/Contraseña + Google OAuth + Modo Invitado
Detalle del Evento	LISTO	Página SSR con mapa, metadata, botón de compartir
Subir Flyer	LISTO	Carga de imagen + geocodificación + etiquetado de categoría
Perfil	LISTO	Perfil de usuario con gestión de eventos
Búsqueda por Ubicación	EN PROGRESO	Autocompletado de ciudad/venue con re-centrado del mapa
Filtro por Categoría	PENDIENTE	Filtrar el feed por tipo de evento

03 Arquitectura y Stack Tecnológico

Stack Tecnológico

CAPA	TECNOLOGÍA	JUSTIFICACIÓN
Framework	Next.js 15 (App Router)	SSR/SSG híbrido, edge-ready, React Server Components
Lenguaje	TypeScript (strict)	Seguridad de tipos, DX, confianza en refactoring
Base de datos	Supabase (PostgreSQL + PostGIS)	Consultas geoespaciales nativas, potencia SQL, RLS
Autenticación	Supabase Auth	Integrado con la BD, compatible con RLS, soporte OAuth
Almacenamiento	Supabase Storage	CDN integrado, políticas de acceso, transformación de imágenes
Estilos	Tailwind CSS 4	Utility-first, zero-runtime, dark mode nativo
Animaciones	Framer Motion	Física de resortes, soporte de gestos, animaciones de layout
Mapas	Mapbox GL JS	Estilos personalizados, API de geocodificación, soporte 3D
Testing	Vitest + Playwright	Velocidad nativa de Vite, E2E a nivel de navegador
Deploy	Vercel (Edge)	Hosting zero-config para Next.js, CDN global

Patrón Arquitectónico: Screaming Architecture

```
src/
  app/                        # Next.js App Router (solo capa de ruteo)
    auth/login/page.tsx       # Página liviana — delega a componentes del feature
    (main)/page.tsx           # Página del Feed
    event/[id]/page.tsx       # Detalle del evento SSR
    api/events/route.ts       # Endpoints REST

  features/                   # DOMINIOS DE NEGOCIO (el "grito")
    feed/                     # Lo que el usuario ve: descubrimiento de flyers
      components/             # UI: FlyerCard, SwipeContainer, FeedHeader
      hooks/                  # Estado: useFeed, useGeolocation
      services/               # Datos: fetchNearbyEvents
      types/                  # Contratos: FeedEvent, FeedState

    auth/                     # Flujos de autenticación
    events/                   # Creación y gestión de eventos
    profile/                  # Perfiles de usuario
    search/                   # Búsqueda por ubicación y categoría

  shared/                     # Concerns transversales
    components/ui/            # Primitivos del design system
    lib/supabase/             # Fábricas de clientes
    lib/utils/                # Utilidades puras
```

> **Principio clave:** Cuando abrís la carpeta `features/`, inmediatamente entendés lo que la aplicación HACE (feed, auth, eventos, perfil) — no cómo está construida (componentes, hooks, servicios). La estructura de carpetas grita el dominio de negocio.

04 Supabase vs Firebase: Decisión Técnica

La elección de base de datos fue la decisión arquitectónica más crítica para Caves. El valor core de la app es el **descubrimiento de eventos basado en geolocalización** — lo que significa que las consultas geoespaciales no son un nice-to-have, SON el producto.

Supabase ELEGIDO

- **PostGIS nativo:** ST_DWithin, ST_Distance, índices GIST — consultas de proximidad sub-10ms en millones de filas
- **Potencia SQL:** JOINs complejos (eventos + categorías + perfiles), agregaciones, window functions
- **Row Level Security:** Políticas auth-aware a nivel de BD — sin necesidad de middleware backend
- **Open source:** Sin vendor lock-in, auto-hosteable, ecosistema Postgres
- **Generación de tipos:** Tipos TypeScript auto-generados desde el schema
- **Migraciones SQL reales:** Versionadas, revisables, con rollback seguro
- **Costo:** Free tier generoso, pricing predecible por proyecto

Firebase RECHAZADO

- **Sin geo nativo:** Firestore requiere workaround con GeoHash — impreciso, complejo, no puede ordenar por distancia
- **Limitaciones NoSQL:** Sin JOINs, sin agregaciones, denormalización requerida en todos lados
- **Reglas de seguridad:** Basadas en JSON, difíciles de testear, sin enforcement a nivel SQL
- **Vendor lock:** Propietario de Google, sin self-hosting, exportación de datos dolorosa
- **Sin generación de tipos:** Mantenimiento manual de tipos, riesgo de drift del schema
- **Costo impredecible:** Facturación por lectura/escritura que puede dispararse con uso intensivo de feed
- **Servicios innecesarios:** ML, analytics, messaging que no necesitamos

El Argumento Geoespacial (Factor Decisivo)

CONSULTA	SUPABASE (POSTGIS)	FIREBASE (GEOHASH)
Encontrar eventos dentro de 25km	`ST_DWithin(location, point, 25000)` — círculo exacto, indexado	Query por rango GeoHash — aproximación rectangular, falsos positivos, post-filtrado requerido
Ordenar por distancia	`ORDER BY ST_Distance(...)` — nativo, indexado	No es posible server-side — hay que traer todo, calcular en el cliente y ordenar
Filtro + geo + orden	Una sola consulta SQL con JOIN, WHERE, ORDER BY	Múltiples queries + merge en cliente + orden manual — problema N+1
Performance a escala	Índice GIST: O(log n) sin importar el tamaño del dataset	GeoHash: se degrada con densidad, requiere índices compuestos por filtro

> **Veredicto:** Para un producto location-first, PostGIS no es solo mejor — es otra categoría. Firebase requeriría construir un motor de consultas geográficas sobre una base de datos que no fue diseñada para eso. Supabase nos da los 30+ años de madurez geoespacial de PostgreSQL out of the box.

05 Arquitectura por Features — Detalle

Reglas de Aislamiento de Features

1. **Sin imports cruzados:** Los features nunca importan entre sí. El código compartido vive en `shared/`
2. **Server Components por defecto:** Solo usar `"use client"` cuando la interactividad lo requiere
3. **Colocación:** Tests, tipos y servicios viven junto al código que sirven
4. **Sin barrel exports:** Importar directamente del archivo, no desde `index.ts`

Inventario de Features

FEATURE	COMPONENTES	HOOKS	SERVICIOS	RENDERIZADO
Feed	FlyerFeed, FeedEmpty, FeedHeader, FlyerCard, SwipeContainer, GeolocationPrompt	useFeed, useGeolocation	feedService	Cliente (geolocation + swipe)
Auth	LoginForm, SignupForm, OAuthButton, AuthGuard	useAuth	authService	Servidor + Cliente híbrido
Events	EventDetail, EventMap, UploadForm, ImageUpload, AddressInput, CategorySelect	—	eventsService, uploadService	SSR (detalle), Cliente (upload)
Profile	ProfileCard, EditForm, AvatarUpload, UserEventsList, SignOutButton	useProfile	profileService	Servidor + Cliente híbrido
Search	SearchBar, LocationPicker, CategoryFilter	useSearch	searchService	Cliente (interactivo)

Estrategia de Renderizado

RUTA	ESTRATEGIA	RAZÓN
/ (Feed)	Client Component	Requiere API de geolocalización + manejo de gestos táctiles
`/event/[id]`	Servidor (SSR)	Crítico para SEO — meta tags OG para compartir en redes
`/upload`	Client Component	Carga de imagen + estado de formulario + geocodificación
`/auth/*`	Servidor + Cliente	Render inicial rápido, hidratación para interacción del form
`/profile`	Servidor + Cliente	Fetch de datos estático, hidratación para acciones de edición

Flujo de Autenticación

MÉTODO	FLUJO	RUTAS PROTEGIDAS
Email/Contraseña	Supabase Auth → cookie de sesión → proxy valida	/upload, /profile, /profile/edit
Google OAuth	Supabase OAuth → callback → cookie de sesión	Igual que arriba
Modo Invitado	Cookie `guest_mode=true` → proxy permite acceso (24h)	Feed, Búsqueda, Detalle de Evento (solo lectura)

06 Modelo de Datos

Relación de Entidades

```
profiles (1) ──────< events (N)     "Un usuario crea muchos eventos"
categories (1) ─────< events (N)     "Un evento pertenece a una categoría"

┌──────────────┐     ┌──────────────────────────────────────┐
│  profiles    │     │  events                              │
├──────────────┤     ├──────────────────────────────────────┤
│ id (PK/FK)   │────>│ user_id (FK)                         │
│ username     │     │ category_id (FK)                     │
│ avatar_url   │     │ title, description                   │
│ bio          │     │ flyer_url (Supabase Storage)         │
│ role         │     │ venue_name, venue_address            │
│ city         │     │ location (PostGIS POINT — lng, lat)  │
└──────────────┘     │ date, time_start, time_end           │
                     │ price, currency, external_url        │
┌──────────────┐     │ status (draft|active|cancelled|past) │
│ categories   │────>│ views_count                          │
├──────────────┤     └──────────────────────────────────────┘
│ id (PK)      │
│ name, slug   │     Índices PostGIS:
│ icon         │     • GIST en location → O(log n) proximidad
└──────────────┘     • B-tree en (status, date) → filtrado de feed
```

Consulta Clave: Eventos Cercanos

```
SELECT e.*, c.name AS category_name,
       ST_Distance(e.location, ST_MakePoint(lng, lat)::geography) AS distance_m
FROM events e
JOIN categories c ON c.id = e.category_id
WHERE e.status = 'active'
  AND e.date >= CURRENT_DATE
  AND ST_DWithin(e.location, ST_MakePoint(lng, lat)::geography, 25000)
ORDER BY distance_m ASC, e.date ASC
LIMIT 20;
```

Row Level Security (RLS)

TABLA	SELECT	INSERT	UPDATE	DELETE
profiles	Todos	Auto (trigger de auth)	Solo perfil propio	—
events	Solo eventos activos	Usuarios autenticados	Solo eventos propios	Solo eventos propios
categories	Todos	—	—	—

Buckets de Almacenamiento

BUCKET	ACCESO	TAMAÑO MÁX.	TIPOS PERMITIDOS
flyers	Público (CDN)	5 MB	JPEG, PNG, WebP
avatars	Público (CDN)	2 MB	JPEG, PNG, WebP

07 Estrategia de Testing

Pirámide de Testing

168	17	15	80%
TESTS UNITARIOS (VITEST)	SPECS E2E (PLAYWRIGHT)	ARCHIVOS DE TEST	COBERTURA OBJETIVO

Cobertura de Tests Unitarios

CATEGORÍA	ARCHIVOS TESTEADOS	TESTS	QUÉ SE CUBRE
Utilidades	cn, format, geo	53	Merge de clases, formateo de fechas/precios, geolocalización, cálculo de distancias
Servicios	feed, auth, events, upload, profile	50	Llamadas API, manejo de errores, flujos de auth, parseo de form data, redirects
Hooks	useFeed, useAuth, useGeolocation, useProfile	40	Manejo de estado, suscripciones, paginación, manejo de permisos
Componentes	LoginForm, FeedHeader, FeedEmpty	25	Renderizado, interacciones de usuario, navegación, comportamiento de menú

Cobertura de Tests E2E

SPEC	TESTS	FLUJOS CRÍTICOS CUBIERTOS
login.spec.ts	6	Visibilidad del logo, acceso como invitado con cookie, renderizado de botones
feed.spec.ts	5	Carga del feed como invitado, renderizado del header, expandir/colapsar tarjetas, scroll horizontal
navigation.spec.ts	6	Abrir/cerrar menú, links de navegación, atajo de upload

Herramientas de Testing

HERRAMIENTA	PROPÓSITO	CONFIGURACIÓN
Vitest 4.x	Tests unitarios y de componentes	Entorno jsdom, alias de paths, setup global
Testing Library	Renderizado de componentes	Soporte React 19, user-event para interacciones
Playwright	Testing E2E en navegador	Viewport mobile (375×812), perfil de dispositivo Pixel 5

Comandos de Test

COMANDO	DESCRIPCIÓN
`pnpm test`	Ejecutar los 168 tests unitarios
`pnpm test:watch`	Modo watch para desarrollo
`pnpm test:coverage`	Ejecutar con reporte de cobertura V8
`pnpm test:e2e`	Ejecutar suite E2E de Playwright

08 Progreso Actual y Roadmap

Fase 1 — MVP Core 85% COMPLETO

Setup del Proyecto e Infraestructura

Next.js 15, Supabase, Tailwind CSS 4, pipeline de deploy en Vercel, TypeScript strict mode, path aliases, ESLint.

Sistema de Autenticación

Email/contraseña + Google OAuth + modo invitado. Protección de rutas basada en proxy. Manejo de sesión vía cookies SSR de Supabase.

Schema de Base de Datos y Migraciones

Schema PostGIS con events, profiles, categories. Índices GIST para consultas geoespaciales. Políticas RLS. Seed data de categorías.

Feed de Flyers (Rediseño de UI)

Grilla masonry de 3 columnas con swipe horizontal por páginas (Framer Motion). Header transparente con menú hamburguesa. Dots indicadores de página. Expansión fullscreen de poster con animaciones de resorte.

Página de Detalle del Evento (SSR)

Renderizado en servidor con meta tags OG. Integración con mapa. Botón de compartir. Tracking de vistas.

Formulario de Subida de Flyer

Carga de imagen a Supabase Storage. Geocodificación de dirección. Selección de categoría. Selector de fecha/hora. Preview antes de publicar.

Suite de Tests

168 tests unitarios + 17 specs E2E. Vitest + Playwright configurados. Umbral de cobertura del 80% enforced.

Pantalla de Login (Diseño Figma)

Diseño dark minimalista siguiendo prototipo de Figma. Botones de Invitado, Iniciar Sesión y Google. En refinamiento.

Fase 2 — Pulido (Siguiente)

- Búsqueda de ubicación con autocompletado
- Filtrado por categoría en el feed
- Funcionalidad de compartir (Web Share API)
- Optimización de imágenes (compresión, conversión WebP)
- Estados de carga, error boundaries, estados vacíos
- Manifiesto PWA + service worker

Fase 3 — Crecimiento (Futuro)

- Guardados / eventos favoritos
- Dashboard de analytics de vistas
- Notificaciones push para eventos guardados
- Herramientas de moderación para admins
- Features premium para promotores (flyers fijados, analytics)

09 Desglose de Inversión de Tiempo

TAREA	HORAS	%	ENTREGABLES
Setup del Proyecto y Configuración	0.5h	6%	Next.js, Supabase, Tailwind, TypeScript, ESLint, Vercel
Schema de BD y Migraciones	0.5h	6%	Tablas PostGIS, índices, RLS, seeds, generación de tipos
Sistema de Autenticación	1.0h	13%	Login, signup, OAuth, modo invitado, proxy, manejo de sesión
Feature del Feed (UI + Lógica)	1.5h	19%	Layout de grilla, swipe horizontal, animaciones, geolocalización
Feature de Eventos	1.0h	13%	Página de detalle (SSR), formulario de upload, manejo de imágenes, geocodificación
Feature de Perfil	0.5h	6%	Página de perfil, formulario de edición, upload de avatar, lista de eventos
Rediseño UI (Figma → Código)	1.0h	13%	Pantalla de login, grilla del feed, navegación del header, tema dark
Suite de Tests	1.5h	19%	168 tests unitarios, 17 specs E2E, configuración de CI
DevOps y Corrección de Bugs	0.5h	6%	Fix de Turbopack, aislamiento de lockfile, pipeline de deploy
TOTAL	**8.0H**	**100%**	

Métricas de Velocidad

9.1	21	5	6
ARCHIVOS / HORA	TESTS / HORA	FEATURES / 8H	PRS MERGEADOS

Nota sobre Desarrollo Asistido por IA: Este proyecto aprovecha Claude Code como herramienta de pair-programming con IA para generación de código, escritura de tests y decisiones arquitectónicas. El desarrollador humano (Alvaro Sepulveda / Craftia) provee toda la dirección creativa, supervisión arquitectónica y aprobación final. La IA acelera la ejecución; la experiencia humana dirige las decisiones.

— Fin del Reporte —

Elaborado por **Craftia** — Consultoría de IA y Desarrollo

24 de marzo de 2026